



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(An Autonomous Institution Affiliated to Anna University, Chennai)

SATHYAMANGALAM - 638 401

Module Test - II - May - 2026

II Semester

22AI206/22AM206/22CS206/22IT206 & DIGITAL COMPUTER ELECTRONICS

1	(i)	An embedded controller calculates net weight by subtracting container weight from total weight. Engineers observe incorrect display outputs when negative values occur. The system depends on sign and zero flags for result validation but misinterprets them. This leads to faulty decisions during operation. Determine how status flag misinterpretation affects arithmetic computation outcomes and recommend a method to ensure correct evaluation of subtraction results. (3 Marks - [An/P, 2])
2	(ii)	A digital energy meter calculates total consumption by adding power readings stored in memory. Engineers observe delays and occasional incorrect totals during rapid updates. The instruction format requires more cycles for memory-based operands. The system must maintain accuracy while improving speed. Evaluate how operand selection affects arithmetic instruction performance and determine its impact on execution speed and accuracy in the given system. (2 Marks - [An/P, 1])
3	(i)	A microcontroller uses NOT operations to invert signal bits for control purposes. Engineers observe unexpected behavior when inversion is applied repeatedly without tracking original values. The system does not maintain reference states for comparison. This leads to incorrect control decisions. Analyze how repeated NOT operations affect signal interpretation and determine a method to maintain correct control logic. (3 Marks - [An/C, 2])
4	(ii)	A sensor interface circuit uses AND operations to filter unwanted noise bits from incoming data. Engineers observe that some valid signals are also being removed during processing. The system applies masking without carefully selecting mask values. Accurate signal extraction is required for proper system operation. Analyze how improper masking using AND operations affects data filtering and determine how correct mask selection ensures accurate signal extraction. (2 Marks - [An/P, 1])
5	(i)	A robotic control system calculates displacement using sequential additions and subtractions. Engineers notice cumulative errors when multiple operations are executed continuously. The system does not verify carry and overflow conditions consistently. This leads to propagation of incorrect values over time. Analyze how ignoring carry and overflow conditions affects sequential arithmetic operations and determine a strategy to maintain computational accuracy. (3 Marks - [An/C, 2])

6	(ii)	A microcontroller in a temperature monitoring system adds incoming sensor readings using an accumulator. During testing, incorrect results appear when large values are processed continuously. The system relies on status flags to detect abnormal conditions but fails to respond correctly. Engineers suspect issues in arithmetic execution and flag handling. Analyze how improper handling of arithmetic instructions and status flags leads to incorrect results, and determine the role of operand type in influencing execution accuracy. (2 Marks - [An/P, 1])
7	(i)	An embedded encryption system uses XOR operations to encode and decode data. Engineers notice incorrect outputs when the same key is not consistently applied. The system relies on XOR properties for reversible transformations. Inconsistent key usage disrupts the encoding process. Determine how XOR operation properties influence data encoding and recommend a method to ensure consistent and reversible transformations. (3 Marks - [An/P, 2])
8	(ii)	A digital control system uses OR operations to combine multiple control signals into a single output. Engineers observe unintended activation of certain functions due to overlapping signal conditions. The system does not properly evaluate input combinations before applying logic operations. Reliable control output must be ensured. Evaluate how improper use of OR operations affects control signal integrity and determine how logical conditions should be structured to avoid unintended outputs. (2 Marks - [E/P, 1])
9	(i)	A data buffering system uses increment operations to move through memory locations and decrement operations to release processed data. Engineers observe data inconsistencies when buffer limits are reached. The system does not properly handle boundary conditions and flag states during these operations. This leads to overwriting or skipping data. The system must maintain data integrity while ensuring efficient buffer management. Analyze how improper handling of increment and decrement instructions affects buffer management and propose a strategy to ensure reliable and efficient operation. (5 Marks - [E/P, 2])
10	(i)	An embedded automation system integrates arithmetic and logic instructions to control multiple processes. Engineers observe inconsistent behavior due to improper handling of data dependencies and instruction order. The system processes operations without ensuring correct data flow between steps. This results in incorrect outputs and reduced efficiency. Evaluate how improper handling of data dependencies and instruction sequencing affects system behavior and develop a strategy to ensure consistent and efficient operation. (5 Marks - [E/P, 2])
11	(i)	A control system in an industrial machine uses counters to manage task execution cycles. Increment instructions increase cycle count, while decrement instructions track remaining operations. Engineers notice incorrect cycle completion detection due to improper flag usage. The system occasionally terminates tasks prematurely or continues beyond required cycles. The design must ensure precise control over execution cycles. Evaluate how improper use of increment and decrement instructions affects cycle control and develop a method to ensure accurate task execution management. (5 Marks - [An/C, 2])

12	(i)	A signal processing unit performs arithmetic transformations on data and uses logic operations for filtering and condition checking. Engineers observe incorrect outputs due to improper interaction between operations. The system does not validate intermediate values and executes instructions inefficiently. This leads to increased processing time and inaccurate results. Analyze how improper interaction between arithmetic and logic instructions affects system performance and propose a method to improve both accuracy and efficiency. (5 Marks - [E/C, 2])
13	(i)	A low-level embedded system performs bit manipulation for device interfacing using rotate instructions. Engineers observe inconsistent device responses due to incorrect bit positioning. The system performs multiple rotations without validating intermediate register values. Lack of proper control over rotation direction and carry flag leads to unpredictable outputs. Evaluate how improper control of rotate operations affects device interfacing and develop a method to ensure consistent and accurate bit manipulation. (5 Marks - [E/P, 2])
14	(i)	A real-time embedded system executes complex operations using arithmetic and logic instructions. Engineers observe performance degradation due to inefficient use of instruction cycles, poor resource utilization, and lack of optimization. The system frequently accesses memory and does not effectively use registers. Instruction dependencies also cause delays. The system must improve throughput while maintaining accuracy. Formulate an optimized execution strategy that improves instruction efficiency, reduces latency, and enhances throughput in the system. (5 Marks - [An/C, 2])
15	(i)	A data transmission unit rotates data bits before sending them to match protocol requirements. Engineers notice incorrect outputs when repeated rotations are applied. The system does not distinguish between logical shift and rotate operations and fails to track carry flag changes. This leads to data corruption during transmission. Analyze how improper distinction between rotate and shift operations affects data integrity and propose a strategy to ensure correct transmission. (5 Marks - [An/C, 2])
16	(i)	A microprocessor-based system processes continuous data streams using arithmetic and logic instructions. Engineers observe reduced responsiveness due to high latency and inefficient instruction scheduling. The system does not balance workload effectively across available resources. This leads to bottlenecks in execution. Analyze how poor instruction scheduling affects system responsiveness and propose a strategy to improve execution efficiency and scalability. (5 Marks - [An/P, 2])

17	(i)	A processor executing memory reference instructions in a real-time monitoring system begins to produce inconsistent outputs during load and store operations. The system uses indirect addressing to handle large datasets, but engineers notice that incorrect values are being fetched intermittently. Detailed analysis reveals that the issue may stem from improper address resolution, timing mismatches between CPU and memory, or incorrect sequencing of instruction execution stages. Since the system is used in critical infrastructure, identifying the exact fault is essential to prevent data corruption and ensure reliable operation. Identify the possible causes of incorrect data retrieval in memory reference instructions and recommend corrective measures. (3 Marks - [E/P, 2])
18	(ii)	A high-performance embedded controller is being designed for an industrial automation system where multiple sensors continuously generate data that must be accessed from memory with minimal delay. The instruction format is constrained by limited word size, forcing the designer to carefully choose between direct and indirect addressing modes. Direct addressing allows faster execution but restricts addressable memory range, whereas indirect addressing introduces additional memory access cycles but enables flexible data handling across a larger memory space. The system must maintain real-time responsiveness while supporting scalability for future expansion. The designer must evaluate performance, hardware complexity, and execution latency before finalizing the addressing mechanism. Formulate a strategic approach to select an addressing mode that optimizes performance while ensuring scalability and flexibility in memory access. (2 Marks - [U/M, 1])
19	(i)	A system integrates arithmetic operations and flag-based decision-making to control execution flow. The engineer must connect how computation results affect flags and how flags influence program behavior. The system depends on accurate coordination between these elements. Relate flag updates to program execution behavior. (3 Marks - [U/P, 2])
20	(ii)	A processor fails to correctly execute conditional instructions due to incorrect flag values stored in the status register. Engineers observe that flags are not updated properly after arithmetic operations, leading to incorrect execution paths. The system must be analyzed to identify the issue and restore proper functionality. Examine the issue and identify causes of incorrect flag behavior. (2 Marks - [U/M, 1])
21	(i)	A computing system executes a memory reference instruction using indirect addressing where the address field of the instruction does not directly point to the operand but instead points to a memory location that stores the effective address. The engineer must analyze the sequence of operations required to resolve the final operand address. The process involves multiple memory accesses and intermediate address retrieval steps. Accurate computation of the effective address is critical to ensure correct execution of arithmetic and data transfer instructions within the processor. Determine the sequence of operations involved in calculating the effective address in indirect addressing and justify each step logically. (3 Marks - [An/M, 2])

22	(ii)	A computer architecture researcher is analyzing how memory reference instructions are executed within a CPU pipeline involving registers, ALU, control unit, and memory modules. Each instruction consists of an opcode and an address field, and execution involves sequential stages such as fetch, decode, and execute. The researcher must understand how these components interact dynamically to perform operations like load, store, and arithmetic computations using memory operands. The system behavior depends on correct coordination between instruction fields and hardware units. The researcher must integrate these elements to model the complete execution flow of memory-based operations. Integrate the roles of instruction format and CPU components to infer how memory reference instructions are executed in a system. (2 Marks - [An/P, 1])
23	(i)	A processor design team must decide which flags to include in the status register for efficient decision-making in control systems. Including more flags increases flexibility but adds complexity. The system must support accurate branching while maintaining performance. Engineers must evaluate which flags are essential for system operation. Formulate a strategy for selecting flags in processor design. (3 Marks - [R/M, 2])
24	(ii)	A high-speed data processing unit uses status flags to monitor arithmetic results and guide execution decisions. Increasing the number of flags improves condition tracking but may slow down processing due to additional hardware complexity. The system must balance performance with accurate condition monitoring. Engineers must evaluate how flag usage affects scalability and efficiency in large-scale systems. Analyze how the number of flags influences system performance and scalability. (2 Marks - [E/M, 1])
25	(i)	A computing system performs operations using register reference instructions where data is manipulated directly within CPU registers without accessing memory. The system uses accumulator and general-purpose registers to perform arithmetic and logical operations efficiently. Engineers must evaluate how these instructions improve system performance and reduce execution time. Evaluate the effectiveness of register reference instructions. (5 Marks - [E/P, 3])
26	(i)	A computing system processes large datasets using both direct and indirect addressing mechanisms. Direct addressing is faster but limited, while indirect addressing supports complex data structures with additional overhead. Engineers must evaluate trade-offs to design an efficient system. Analyze the trade-offs between addressing modes. (5 Marks - [An/C, 3])
27	(i)	A processor executes register instructions involving multiple steps controlled by the control unit. The engineer must outline the sequence of execution. Construct the execution flow. (5 Marks - [R/M, 3])

28	(i)	A system combines extended register operations with indirect addressing to handle dynamic data structures efficiently. The engineer must relate these concepts to system functionality. Relate the concepts. (5 Marks - [An/C, 3])
29	(i)	A system uses multiple jump and call instructions to manage execution flow in complex applications. While these provide flexibility, they may introduce overhead and reduce readability. Engineers must evaluate trade-offs. Analyze the trade-offs. (5 Marks - [An/C, 3])
30	(i)	A large-scale application uses stack organization to manage function calls, recursion, and interrupt handling. While stacks provide structured execution flow, they consume memory and may cause overflow in deep recursive calls. The system must balance efficient execution with memory constraints. Engineers must evaluate trade-offs between stack usage, performance, and scalability. Analyze the impact of stack organization on system performance and scalability. (5 Marks - [An/C, 3])
31	(i)	A processor executes call and return instructions requiring saving and restoring execution addresses using stack memory. The engineer must outline the execution process. Develop the sequence of operations. (5 Marks - [E/F, 3])
32	(i)	A processor executes a sequence of push and pop operations where multiple data values are stored and retrieved from the stack. The engineer must model how the stack pointer changes and how data is managed during execution. Accurate modeling ensures correct execution flow. Determine how stack operations affect stack pointer movement. (5 Marks - [An/M, 3])